SkyBalance AVL
Sistema de Gestión Aérea

La aerolínea **SkyBalance** enfrenta un problema importante y necesita gestionar cientos de vuelos activos en tiempo real sin que las búsquedas, inserciones o cancelaciones pierdan eficiencia. Cada nueva ruta modifica la estructura operativa de la empresa, y cualquier desbalance puede traducirse en decisiones tardías, sobrecostos o inconsistencias en las conexiones entre ciudades. Su misión será diseñar e implementar un sistema que resuelva este problema utilizando como núcleo un **Árbol AVL**, garantizando equilibrio estructural y rendimiento.

En este proyecto, ustedes no solo desarrollarán un árbol que inserta y rota nodos. Construirán un sistema dinámico donde la posición de cada nodo (vuelo) dentro del árbol afectará precios, promociones, pasajeros y prioridades. Cada rotación alterará la jerarquía de rutas; cada cancelación podrá eliminar ramas completas; cada inserción impactará métricas globales de eficiencia. Además, deberán integrar recorridos permanentes (profundidad y anchura), historial de estados con pilas, validación formal de la propiedad AVL y visualización estructural en tiempo real.

El reto consiste en transformar una estructura de datos en el cerebro operativo de una aerolínea simulada. Deberán garantizar balance, coherencia geográfica entre rutas, persistencia estructural y análisis matemático de desempeño. Al finalizar, no solo demostrarán que el árbol se mantiene balanceado, sino que serán capaces de explicar cómo la estructura influye en la eficiencia del sistema y por qué un AVL representa una solución frente a alternativas no balanceadas.

A continuación, se presentan los requisitos del proyecto.

1 Persistencia Inteligente y Reconstrucción Estructural

1.1 Lectura y administración del árbol

El árbol inicial se debe cargar desde un archivo JSON que es seleccionado por el usuario a través del navegador de archivos, no es permitido tener una ruta fija en el código. Cuando se lee el archivo el árbol debe reconstruirse a partir de dos tipos posibles (los dos deben existir):

- Topología: se debe respetar la topología descrita en el JSON con padres e hijos.
- Inserción: insertar uno a uno los vuelos de una lista encontrada en el JSON.

En el caso que sea el de inserción, se debe insertar en el AVL (con balanceo automático) y paralelamente insertar en un BST. Al final de la lectura se debe mostrar el gráfico de cómo quedaron los dos y mostrar las propiedades: raíz, profundidad, cantidad de hojas de cada

árbol. Esto puede ser visto en una ventana adicional, manteniendo el AVL principal en una interfaz única para su gestión y aplicación de las demás funcionalidades.

1.2 Comportamiento del árbol ante gestión de nodos

La administración del árbol deberá permitir a través de la interfaz gráfica la creación, modificación o eliminación de nodos (vuelos). Esto generará un balanceo automático del árbol. Cada adición, eliminación o modificación de nodos deberá ser almacenado de tal manera que pueda hacerse un retroceso de la acción (a modo de ctl+z) mediante un botón.
Al igual, cuando un vuelo es cancelado, el nodo que corresponde a ese vuelo deberá ser eliminado en conjunto con todos sus descendientes, generando un balanceo masivo en el árbol, esta acción de cancelación es diferente a la eliminación, la eliminación solo afecta a un nodo, mientras la cancelación de vuelo afecta el nodo correspondiente al vuelo y a toda su decendencia.

1.3 Guardado del Estado en JSON

El sistema deberá permitir:

- Exportar el árbol completo a JSON
- Guardar:
  - Estructura jerárquica de todo el árbol.
  - Alturas
  - Factores de equilibrio
  - Precio base y final
  - Pasajeros
  - Promociones
  - Alertas
  - Prioridades
  - Otro que consideren necesario para la ejecución del proyecto.

Restricción: No se permite guardar únicamente la lista de vuelos. Debe serializarse la estructura real del árbol.

2 Sistema de Versionado Persistente

Además de la pila de retroceso de acción del punto anterior, se realizará un versionado del árbol en el momento que el usuario lo desee, dándole un nombre específico. El usuario puede:

- Guardar una versión por ejemplo como “Simulación Alta Demanda”
- Restaurar versiones específicas. Es decir, se debe mantener una referencia a cada versión guardada por el usuario y cargarse mediante su selección.

3 Simulación de Concurrencia (Inserciones Simultáneas)

Aunque sea un sistema de un usuario usándolo altiempo, se simularán:

- N flujos de inserción paralelos.
- Cola de solicitudes de vuelo.

Se debe implementar una interfaz que permita programar inserciones, sin ejecutarlas en ese momento, y luego un botón que procese la cola y que se muestren los movimientos en el árbol con cada inserción. Para esto se debe tener:

- Una Cola (Queue) de inserciones pendientes.
- Procesador que:
    - Toma solicitudes
    - Las inserta una por una
    - Muestra conflictos si afectan balance crítico.

4 Métricas Analíticas del Árbol

El sistema debe calcular en tiempo real:

- Altura actual.
- Número total de rotaciones realizadas categorizadas por su tipo.
- Número total de cancelaciones masivas.
- Recorridos en anchura y profundidad.
- Cantidad de hojas.

5 Rebalanceo Diferido (Modo Estrés)

El sistema puede entrar en modo estrés donde el balanceo no se realiza de manera automática, las operaciones (agregar, editar, eliminar, cancelar) siguen su curso pero sin realizar balanceo automático. Cuando se activa:

- Se permiten operaciones sin balance inmediato.
- El árbol puede degradarse.
- Se muestra visualmente como se deforma.

Luego el usuario puede regresar al modo “Rebalanceo Global”. En este modo el sistema debe:

- Recorrer el árbol.
- Detectar nodos desbalanceados.
- Aplicar rotaciones necesarias en cascada.

- Medir cuánto costó estructuralmente la reparación (estadísticas de giros según categoría).

6 Sistema de Penalización por Profundidad Crítica

Si un nodo supera una profundidad específica (dada por el usuario desde antes de cargar el JSON, pero modificable en cualquier momento de la ejecución del sistema), Entonces:

- Se activa bandera NodoCritico
- Se incrementa precio en 25% al costo inicialmente dado por el usuario. Si el usuario modifica la profundidad límite en cualquier momento, el sistema debe calcular los valores de los vuelos nuevamente con base en la regla del 25% incremento/decremento.
- Este nodo debe tomar un color diferencial a los demás cuando se encuentre por debajo de la profundidad parametrizada.

7 Sistema de Auditoría AVL

Debe existir un botón “Verificar Propiedad AVL”. Este botón solo estará disponible en el modo estrés. El sistema debe:

1. Recorrer todo el árbol
2. Verificar:
    - Factor de Balanceo ∈ {-1,0,1}
    - Altura correcta
3. Mostrar reporte detallado personalizado.
4. Indicar nodos inconsistentes.

8 Eliminación Inteligente por Impacto Económico

El sistema debe contar con un botón “Eliminar Nodo de Menor Rentabilidad”. En este caso el sistema debe:

1. Calcular rentabilidad = pasajeros × precioFinal – promoción (si aplica) + penalización (si aplica)
2. Encontrar el nodo de menor rentabilidad. En caso de encontrar más de uno con igual menor valor, se debe tomar el más lejano a la raíz, y si persiste la igualdad, se tomará el de código más grande.
3. Cancelar ese nodo, considerando que la cancelación afecta a toda la subrama.
4. Rebalancear.

Entregables

- Documentación (manual de usuario y manual técnico).
- Video explicativo de ls solución y arquitectura del sistema (en un segundo idioma), todas las personas del equipo deben participar y deben estar en primer plano.
- Viudeotutorial: video explicativo del sistema, en primer plano estará el código e interfaz gráfica ejecutándose, en segundo plano deben estar las personas (todos participan).
- Código debidamente versionado en una plataforma que soporte GIT, donde se evidencie los aportes de cada integrante del equipo. El código debe estar debidamente documentado en inglés.

Fecha de sustentación: martes 24 de marzo o martes 7 de abril (se acuerda en clase). En cualquier fecha, la entrega deberá realizarse el día anterior a más tardar a las 8:00 pm. vía correo electrónico. La documentación y enlace a plataforma git deberán estar adjuntos en el correo, el vídeo puede estar cargado en Youtube o en Drive, y referenciar el enlace en el cuerpo del mensaje.

Evaluación:

- Funcionalidad: 50% (en equipo).
- Sustentación individual: 50%.

La nota de la sustentación individual no podrá ser menor en una unidad (1) de la nota de la funcionalidad, en tal caso, la nota de funcionalidad será disminuida hasta el valor de la nota de la sustentación individual para el estudiante en cuestión.